

The Seamless Shopping Experience: Migrating from Monolithic to Microservices Architecture

Assignment #5

Software Systems Architecture

1MEIC02 T3

Gonçalo Gonçalves da Costa Pinto
José Diogo Alves Granja
Manuel Mo
Pedro Rafael Correia Borges
Ricardo Miguel Yang

1. Assessment of the Current Architecture

One year ago, we successfully introduced the **Automated Smart Shopping Cart system** to the market. At the initial stages, when there were only three people in the team, **Monolithic Architecture** seemed to be the most reasonable decision because of fast development and deployment processes. However, the system's operational success and the subsequent growth of our engineering department have outpaced this foundational architecture.

1.1. The “Cloud Services” Monolith

Currently, the backend is implemented using a monolithic approach (the “Cloud Services” hub). All aspects from sensor-based item detection to sophisticated pricing rules, payments, and hardware communication are handled within one large codebase. As a result, each request (from trivial lookups to payments) goes through the same pipeline and shares the same CPU and RAM pool. Besides, when all business logic and hardware dependencies are hardcoded into one codebase, then any changes in the system will cause a recompile and redeployment of the entire system.

1.2. Performance Limitations

The monolithic structure has reached its operational ceiling, with the **Pricing and Promotions Engine** becoming a critical bottleneck. The logic for pricing-related calculations (e.g., membership discounts, promotional bundles, and real-time recommendations) is tightly interwoven with the core cart state. During peak store hours, these high-intensity, synchronous calculations inject significant latency into the system. This manifests as a **1-2 second lag** on the user's mobile app. In a physical retail environment, this delay creates noticeable friction, degrading the seamless “grab-and-go” experience and making the smart cart feel unresponsive.

1.3. Conway’s Law and Organizational Mismatch

According to Conway’s Law [1], *“organizations which design systems... are constrained to produce designs which are copies of the communication structures of these organizations”*. Last year our three-man team was a perfect reflection of our monolithic codebase. Today, the engineering department has grown to twelve engineers (soon expanding to seventeen) organized into specialized domain teams (e.g., Cart/Hardware, Pricing, Mobile, and Payment and Security). The monolithic architecture is now fundamentally at odds with our organizational structure. In other words, our organization no longer matches the monolithic architecture.

1.4. Development Friction and Deployment Contention

Because these newly specialized teams operate within a single, shared codebase, development friction is exceptionally high. Teams frequently block one another. A minor update to the “Recommendations” logic requires a full regression test of unrelated critical systems, such as “Payments” and “Store Alarms”. Furthermore, deployment contention limits our agility. It is impossible to deploy a localized fix or scale a specific module (like the Pricing engine during a flash sale) without redeploying and scaling the entire system. This drastically increases the “blast radius” of any potential bug and stifles our ability to deliver competitive new features rapidly.

2. Proposed Microservices Architecture and System Boundaries

In order to resolve the monolithic bottlenecks and accommodate with the newly expanded organizational structure composed of specialized teams, it is proposed that there be a move towards adopting a **Domain-Driven Microservices architecture** [2]. This redesign decouples the system into highly cohesive, independently deployable units, fundamentally shifting our system's dependencies from compile-time to deploy-time binding.

2.1. System Context and External Integrations

Before detailing the internal microservices, it is essential to define the system's boundaries and its relationship with the broader enterprise environment. Figure 1 illustrates the expanded scope of the **Automated Smart Shopping System**. While the core human actors remain consistent, the system now integrates with two new, critical external platforms:

- **ERP (Enterprise Resource Planning)** [3]: Represented as the "Store ERP / Inventory DB", this is the foundational back-office software managing the company's day-to-day physical operations. It serves as the ultimate source of truth for physical store stock levels, product metadata, and base pricing.
- **CRM (Customer Relationship Management)** [4]: This external system manages interactions and data related to customers over the long term. By integrating with the CRM, our system can access cross-store shopping history and loyalty tiers, enabling new business requirements like personalized, multi-store recommendations.

By establishing strict boundaries between our internal system and these external ERP and CRM providers, we prioritize the quality attribute of **Modifiability**. The system can integrate new CRM analytics or change payment gateways without altering the core shopping logic.

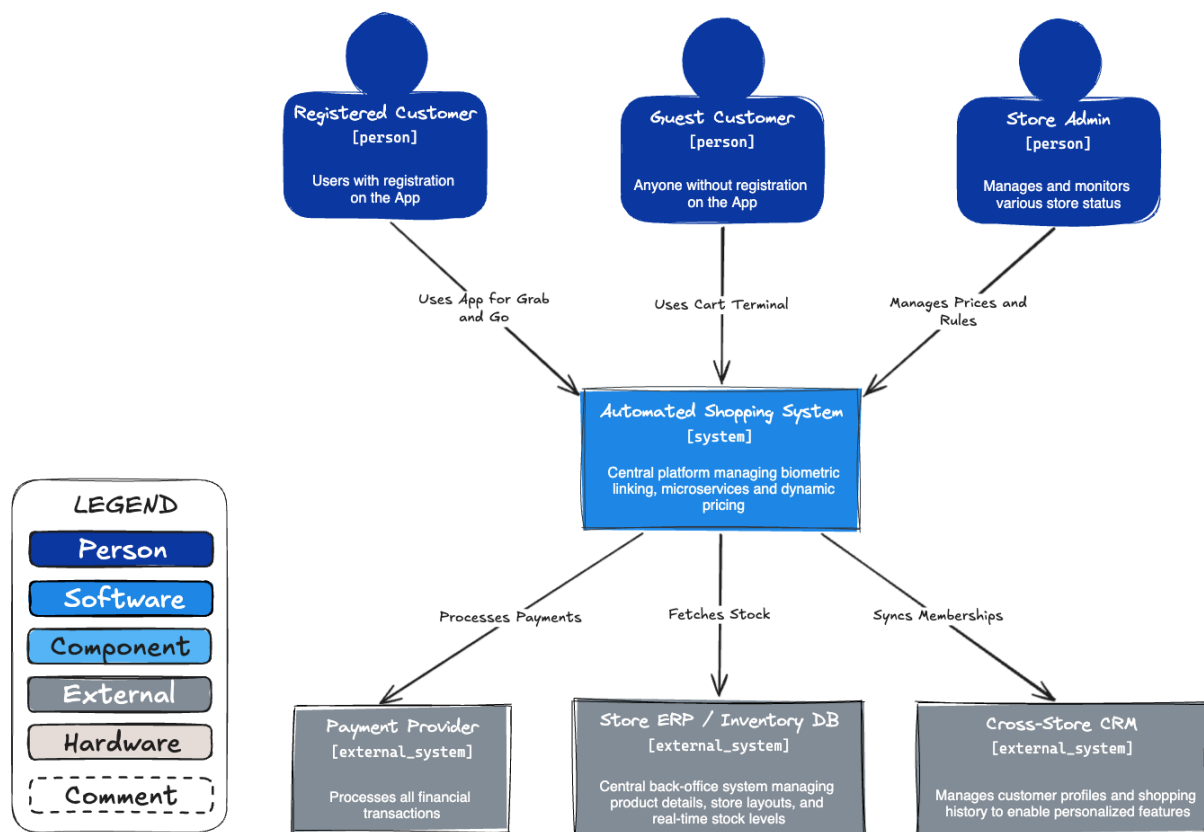


Figure 1: C4 Level 1, System Context Diagram

2.2. Service Boundaries and Container Architecture

The transition to microservices resolves the organizational friction (Conway's Law mismatch) by splitting the monolith into distinct bounded contexts, allowing the newly specialized teams to develop, test, and deploy independently. Figure 2 maps these new boundaries.

The system has been decomposed into five primary microservices, highlighted with the dashed blue box in the figure:

1. **Cart and Session Service:** Manages the active, real-time physical state of the cart, including the RFID (Radio-Frequency Identification) sensors used to detect physical items placed in the basket.
2. **Payment Service:** Orchestrates the final financial transaction and digital receipt generation.
3. **Pricing and Promotions Service:** Owns all financial calculations, discounts, and flash sales.
4. **User and Security Service:** Manages biometric identity, profiles, and AI-based fraud detection.
5. **Recommendation Service:** An isolated analytical engine for cross-selling and budget mode calculations.

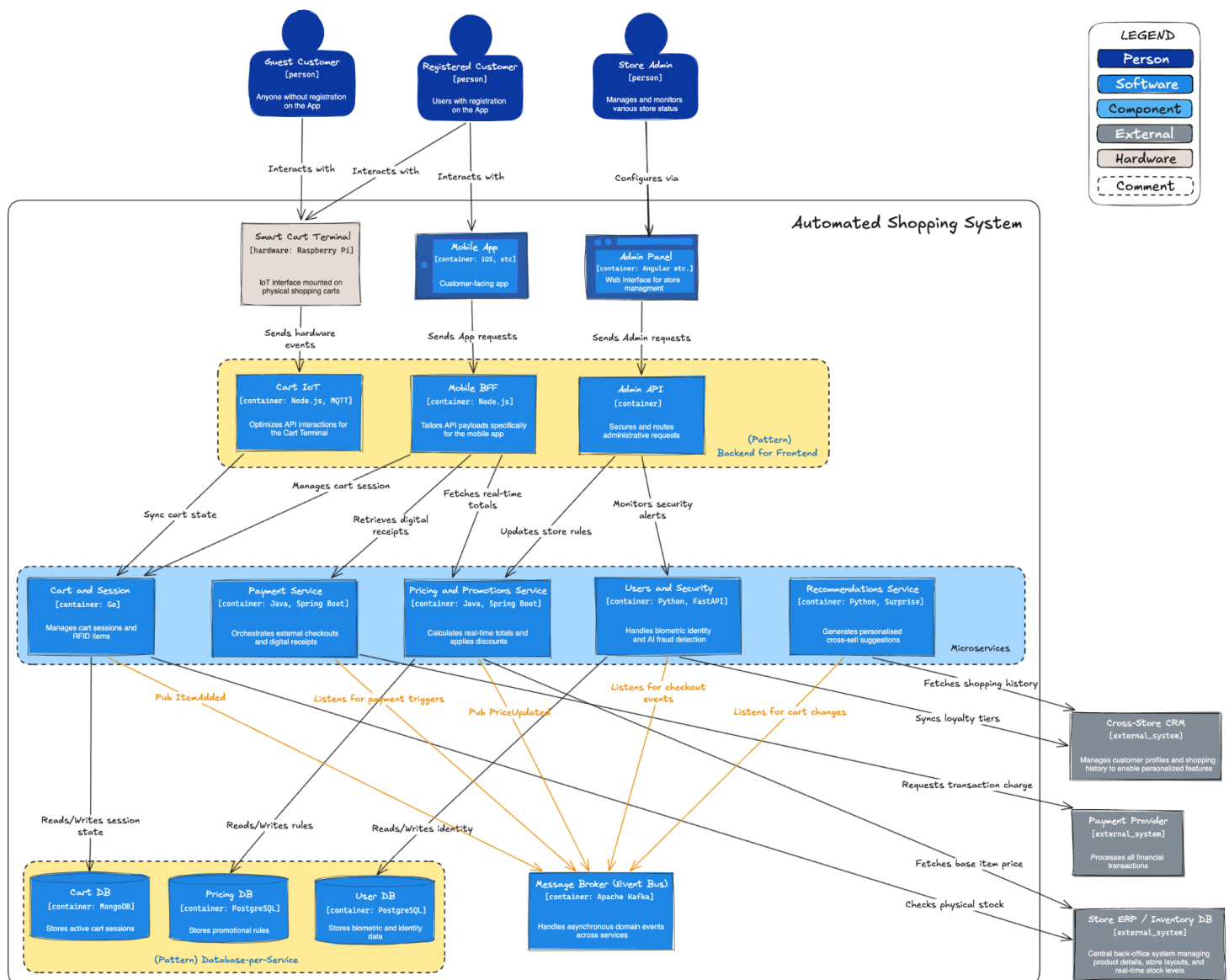


Figure 2: C4 Level 2, Container Diagram

2.2.1. Integration of Cloud Patterns

To facilitate this decomposition, two primary **Cloud Adoption Patterns** [5] were introduced (grouped in yellow dashed boxes in Figure 2):

- **BFF (Backend for Frontend)** [6]: Rather than forcing the Mobile App and the Smart Cart Terminal to consume a one-size-fits-all API, we introduced dedicated BFFs (Mobile BFF and Cart IoT BFF). IoT (Internet of Things) refers to the physical cart terminal acting as a connected device on the network. This pattern enhances **Usability** and **Modifiability**. The mobile team can aggregate data into a single mobile-optimized payload without affecting the hardware team's strict IoT message contracts.
- **Database-per-Service** [7]: To eliminate data coupling, persistent storage was decentralized. The Cart Service utilizes a NoSQL database for rapid session writes, while the Pricing and User services utilize relational SQL databases for strict transactional integrity. This pattern significantly provides **Availability**. If the Pricing database experiences a local failure, the Cart database remains operational, ensuring active shopping sessions do not crash. The inherent trade-off of this pattern is the complexity of distributed data management, necessitating eventual consistency across boundaries.

3. Inter-Service Communication and Event-Driven Architecture (EDA)

The architecture heavily leverages an **Event Broker** (Apache Kafka) to handle communication between the decoupled services. In the container diagram, inter-service communication protocols are differentiated by arrow colors (also to distinguish visually): **black** arrows denote **synchronous** API or external requests, whereas **orange** arrows represent **asynchronous** event-driven messaging [8].

3.1. Synchronous vs. Asynchronous and Eventual Consistency

We employ both communication styles strategically to maximize user experience and system resilience:

- **Asynchronous (Eventual Consistency):** When a user places an item in the cart, the system publishes an event. If the recommendation engine is 5 seconds behind the current cart state, the system still functions perfectly. We accept Eventual Consistency during the active shopping phase, so the cart UI is never blocked waiting for a backend process.
- **Synchronous (Strong Consistency):** On the other hand, synchronous calls are strictly reserved for critical validation points. Checkout authorization and biometric verifications must be strongly consistent, such as we cannot charge a customer the wrong amount or allow an unverified exit.

The selection of communication style is driven by the specific operational context, prioritizing speed and decoupling for general UX, while mandating strict consistency for financial and security checkpoints, as detailed in the comparison in table 1.

Event	Method	Reason
Cart UI (Item adds / Item removes)	Synchronous	A user needs to see the item being added close to instant, so that it betters his experience and removes any possible duplicates.
Payment Authorization	Synchronous	Requires Strong Consistency . We cannot allow a user to exit the store without a verified transaction from the gateway.
Pricing Rule Update	Synchronous	Admin changes to flash sales are pushed directly to the Pricing Service via REST API calls. The system ensures that the rule is persisted to the database and the distributed cache is updated/invalidated within the same request cycle, ensuring immediate consistency before the Admin receives a success confirmation.
Biometric Verification	Synchronous	Security critical. The exit gate remains locked until the User Service provides a real-time “Success” response.
Recommendations Service	Asynchronous	Non-critical path. If the engine is 2 seconds behind the current cart state, the core shopping experience remains unaffected.

Table 1: Sync vs. Async Events

3.2. Main Domain Events

Services publish and subscribe to domain events via the Message Broker to trigger cross-domain logic:

- **ItemAdded / ItemRemoved:** Published by the Cart Service. Consumed by the Pricing Service (to recalculate totals) and the Recommendation Service (to update suggestions).
- **PriceUpdated:** Published by the Pricing Service. Consumed by the Mobile BFF to push real-time UI updates to the user's phone.
- **CheckoutInitiated:** Published by the API Gateway. Consumed by the Payment Service and Fraud Detection systems.
- **PaymentTriggered:** Published by the Payment Service. Consumed by the Store ERP to officially adjust physical stock.

4. Resolving the Pricing and Promotions Bottleneck

The current **1-2 second lag** in cart updates is a major operational concern. Figure 3 zooms into the internal architecture of the Pricing and Promotions Service to demonstrate how decoupling, speed, and flexibility eliminate this bottleneck.

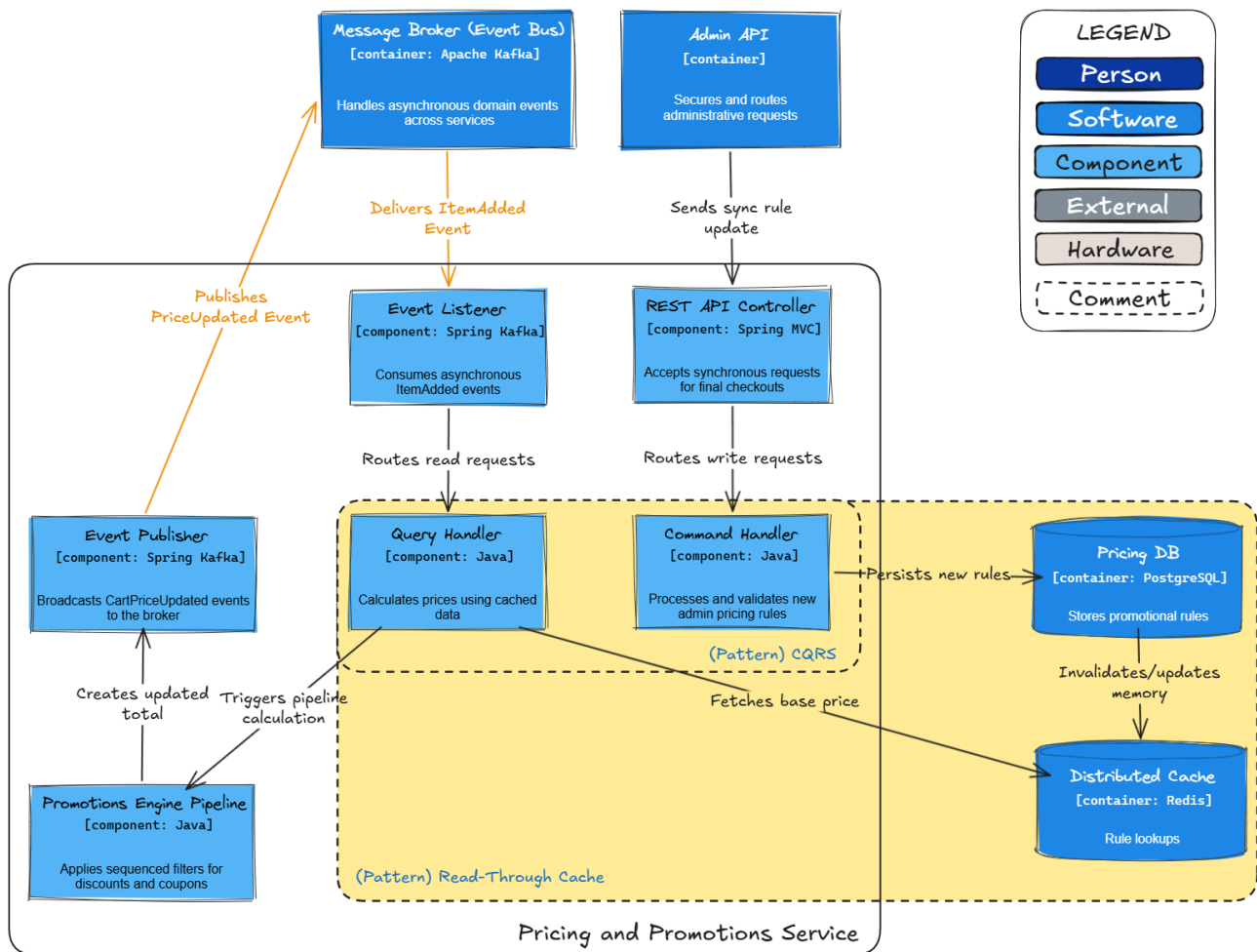


Figure 3: C4 Level 3, Component Diagram

4.1. The CQRS Pattern

The core of the solution is the implementation of the **CQRS (Command Query Responsibility Segregation)** pattern [9], highlighted by the yellow dashed box in Figure 3. CQRS physically separates the logic for reading data from the logic for writing data.

- **Command Handler:** When store managers configure complex flash sales or new promotional rules, these synchronous writes are processed here and persisted to the PostgreSQL Pricing Database.
- **Query Handler:** When the Event Listener consumes an ItemAdded event, it routes strictly to the Query Handler.

This pattern helps resolve the pricing and promotions bottleneck using the Query Handler for high-speed read operations while isolating potentially expensive write operations within the Command Handler.

4.2 Cache Strategy and Trade-offs

To achieve the **Performance** requirement of sub-millisecond cart updates, the Query Handler never queries the PostgreSQL database directly during an active shopping session. Instead, it relies on a **Distributed Cache** (Redis) [10]. This follows the **Read-Through pattern**, where the Query Handler reads pricing and promotion data directly from the cache, while Redis transparently loads missing data from PostgreSQL on a cache miss and stores it for future requests. This ensures that frequently accessed data remains available entirely in-memory while PostgreSQL continues to act as the durable persistence layer. When the Command Handler updates a rule, it asynchronously invalidates and updates this cache.

This guarantees that the active Promotions Engine Pipeline, which applies member discounts and dynamic surge pricing, operates entirely in-memory. The pipeline calculates the new total and utilizes the Event Publisher to broadcast a `CartPriceUpdated` event back to the broker.

Furthermore, this entire EDA and caching strategy relies on run-time binding. The Pricing Service evaluates dynamic logic at run-time by querying Redis, allowing store managers to configure and trigger flash sales instantly without requiring a system restart or a new deployment.

The primary trade-off of this high-performance caching strategy is the risk of **transient stale data**. If a store manager updates a flash sale, there is a microsecond window where a customer might be quoted the old price before the cache invalidates. Given the retail domain, this micro-inconsistency is an acceptable business trade-off to ensure the cart interface remains highly responsive and scalable under peak load. Additionally, introducing Redis as a distributed cache increases **operational complexity**, requiring its own monitoring, replication, and failover strategy. If Redis becomes unavailable, the Query Handler has no fallback path for pricing data, however, this risk is mitigated by deploying Redis with replication and automatic failover monitoring, ensuring that if the primary cache node fails, a replica is promoted automatically without disrupting active shopping sessions. This makes the added operational overhead an acceptable cost for the performance gains achieved.

5. Migration Strategy

Given that the Automated Smart Shopping Cart is actively deployed across multiple high-volume retail locations, a simultaneous total system replacement (commonly referred to as a “Big Bang” deployment) introduces an unacceptable level of operational risk. To guarantee zero downtime and ensure continuity of the customer experience, the system will be migrated using the **Strangler Fig Pattern** [11]. This methodology facilitates the incremental extraction of domains into independent microservices while the legacy monolith remains operational, routing traffic over progressively as confidence in the new architecture is established.

The complete migration schedule is visually detailed in Figure 4.

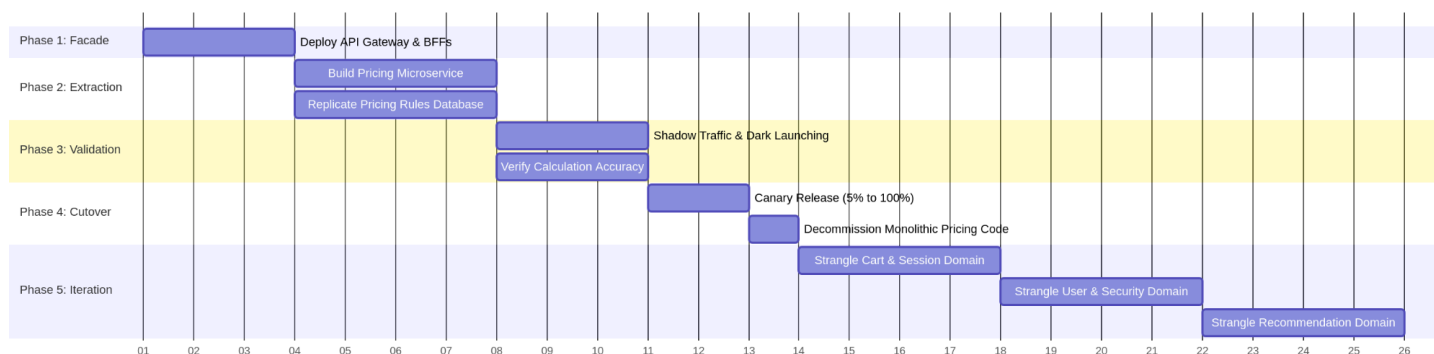


Figure 4: Strangler Fig Pattern Schedule Gantt Diagram

5.1. Phased Migration Schedule

The migration will be executed in a strictly phased sequence, prioritizing the extraction of the most critical system bottleneck: the Pricing and Promotions engine.

- **Phase 1: Facade Implementation (Weeks 1-3)**

Prior to altering any core business logic, the new Admin API Gateway and the BFFs (Backend for Frontend) will be deployed in front of the existing legacy monolith. During this phase, these routing layers act exclusively as a proxy, forwarding 100% of network traffic to the legacy system. This establishes the structural facade required to route traffic dynamically in subsequent phases.

- **Phase 2: Service Extraction (Weeks 4-7)**

Operating entirely behind the scenes, the designated engineering team will construct the new independent Pricing and Promotions microservice, provisioning its dedicated PostgreSQL database and Redis caching layer. The existing promotional rules and base pricing data will be replicated into this new infrastructure.

- **Phase 3: Shadow Traffic and Validation (Weeks 8-10)**

To definitively validate the accuracy of the new CQRS and caching architecture without jeopardizing active retail transactions, the system will utilize Dark Launching and Shadowing. The API Gateway will be configured to duplicate incoming cart pricing requests. The primary request will be processed by the legacy monolith and returned to the physical cart terminal. Simultaneously, the duplicated (shadow) request will be routed to the new microservice. The outputs will be systematically logged and compared to guarantee absolute mathematical parity under production loads.

- **Phase 4: Canary Release and Cutover (Weeks 11-13)**

Following the successful validation of shadow traffic, the cutover process will commence via a Canary Release (Weeks 11-12). The API Gateway will incrementally route live pricing traffic to the new microservice, starting at 5% of active stores. Assuming system telemetry indicates no latency degradation or error spikes, this allocation will be systematically increased to 25%, 50%, and ultimately 100%. Once all pricing traffic is exclusively managed by the new microservice, the redundant pricing logic within the legacy monolith will be officially decommissioned (strangled) and removed from the codebase (Week 13).

- **Phase 5: Iterative Strangling (Weeks 14-25)**

The proven operational framework (Extraction → Shadowing → Cutover) will subsequently be applied to the remaining system domains in a prioritized order: Cart and Session, User and Security, and finally, Recommendations. The process of extracting these services is able to be that much quicker because the foundational infrastructure was already established (e.g., the API Gateway, Kafka, CI/CD, and shadowing testing tools).

5.2. Migration Testing and Quality Assurance

Testing methodologies must evolve to accommodate the distributed nature of the migration process. Relying exclusively on legacy end-to-end (E2E) regression tests is structurally insufficient.

- **Consumer-Driven Contract Testing:** Strict API contracts will be established between the Hardware (Cart) team and the Pricing team. Automated contract validation will be integrated into the CI/CD pipeline to guarantee that architectural iterations within the Pricing service do not disrupt the physical cart terminals.
- **Eventual Consistency Monitoring:** With the introduction of the Apache Kafka message broker, new observability suites will be implemented to verify that asynchronous domain events (e.g., ItemAdded) are reliably consumed and processed within acceptable Service Level Agreements (SLAs), even in the event of transient network partitions.

5.3. Migration Costs and Trade-offs

While the Strangler Fig Pattern guarantees a zero-downtime transition, the specific costs and trade-offs inherent to this strategy also need to be accounted for:

- **Infrastructure Overlap:** During the 25-week migration, we must maintain the legacy monolithic infrastructure while simultaneously spinning up and paying for the new microservices architecture (including the new API Gateway, Kafka cluster, and Redis caches).
- **Compute Overhead during Shadowing:** In Phase 3, utilizing Shadow Traffic means the system will temporarily process every cart pricing request twice. This will effectively double the compute load and database write costs for the pricing domain during weeks 8-10.
- **Opportunity Cost:** Dedicating a significant portion of our expanding engineering team to architectural extraction means a temporary reduction in the delivery of brand-new features, but even with these short-term financial and operational costs, this investment is strictly necessary to resolve the current system bottlenecks and unlock long-term feature velocity.

6. Conclusion

The original monolithic architecture of the Automated Smart Shopping Cart successfully launched the product but ultimately fell victim to Conway's Law as the engineering team expanded. To resolve the resulting development friction and the critical pricing latency, the system has been fundamentally redesigned into a domain-driven Microservices Architecture.

By decoupling the monolith into five specialized services and adopting an EDA, the platform now supports independent team velocity and highly resilient, asynchronous operations. Furthermore, the strategic implementation of specific Cloud Adoption Patterns (namely Backend for Frontend , Database-per-Service, and CQRS combined with distributed caching) has successfully eliminated the performance bottlenecks while guaranteeing strict fault isolation.

Finally, by executing this transition incrementally via the Strangler Fig Pattern, the business ensures a risk-mitigated, zero-downtime migration, establishing a highly scalable foundation capable of sustaining future retail growth.

References

1. Conway, M. E. (1968). How do committees invent. *Datamation*, 14(4), 28-31.
2. Hippchen, B., Schneider, M., Giessler, P., & Abeck, S. (2019). Systematic application of domain-driven design for a business-driven microservice architecture. *International Journal on Advances in Software*, 12.
3. Shehab, E. M., Sharp, M. W., Supramaniam, L., & Spedding, T. A. (2004). Enterprise resource planning: An integrative review. *Business process management journal*, 10(4), 359-386.
4. Kumar, V., & Reinartz, W. (2018). Customer relationship management.
5. Brown, K., Woolf, B., De Groot, C., Hay, C., & Yoder, J. (n.d.). Patterns for developers and architects building for the cloud. *Cloud Adoption Patterns*. Retrieved May 12, 2026, from <https://kgb1001001.github.io/cloudadoptionpatterns/>
6. Brown, K., Woolf, B., De Groot, C., Hay, C., & Yoder, J. (n.d.). Backend for frontend. *Cloud Adoption Patterns*. Retrieved May 12, 2026, from <https://kgb1001001.github.io/cloudadoptionpatterns/Microservices/Backend-For-Frontend/>
7. Messina, A., Rizzo, R., Storniolo, P., Tripiciano, M., & Urso, A. (2016, August). The database-is-the-service pattern for microservice architectures. In *International Conference on Information Technology in Bio-and Medical Informatics* (pp. 223-233). Cham: Springer International Publishing.
8. Brown, K., Woolf, B., De Groot, C., Hay, C., & Yoder, J. (n.d.). Event-driven architecture. *Cloud Adoption Patterns*. Retrieved May 12, 2026, from <https://kgb1001001.github.io/cloudadoptionpatterns/Event-Based-Architecture/Event-Driven-Architecture/>
9. Brown, K., Woolf, B., De Groot, C., Hay, C., & Yoder, J. (n.d.). Command query responsibility separation. *Cloud Adoption Patterns*. Retrieved May 12, 2026, from <https://kgb1001001.github.io/cloudadoptionpatterns/Coexistence-Patterns/Command-Query-Responsibility-Separation/>
10. Kaptosv, L. (2025). Using Redis for caching optimization in high-traffic web applications. *International Journal of Advanced Multidisciplinary Research and Studies*, 5(4), 1714-1722.
11. Brown, K., Woolf, B., De Groot, C., Hay, C., & Yoder, J. (n.d.). Strangler patterns. *Cloud Adoption Patterns*. Retrieved May 12, 2026, from <https://kgb1001001.github.io/cloudadoptionpatterns/Strangler-Patterns/README/>